Primjena mikroupravljačkih sustava

Laboratorijska vježba 1

Upoznavanje platforme: razvojna pločica, memorija, pinovi i rad sa senzorom

Student: **Tomislav Perković** • Datum: **14. 10. 2026.**

Zadatci vježbe

> **1. Stvoriti projekt prema poglavlju 4 i pokrenuti program koji trepće ugrađenom LED-icom na pinu D13. Time se potvrđuje ispravnost alatnog lanca i veze s pločicom.**

Koristi se klasični program za treptanje LED-ice:

```
const uint8_t LED_PIN = 13;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

LED-ica svijetli 500 ms, a zatim je ugašena 500 ms i tako se vrti u beskonačnoj petlji.

> **2. Iz ispisa nakon prevođenja očitati koliko je zauzeto flasha i koliko SRAM-a te vrijednosti upisati u tablicu 8.**

Nakon prevođenja početnog programa Arduino IDE ispisao je sljedeće zauzeće memorije:

Stavka	Vrijednost
Zauzeće flasha nakon zadatka 1 (B i %)	1918 B (6 %)
Zauzeće SRAM-a nakon zadatka 1 (B i %)	184 B (8 %)

Udio flasha računa se od 30.720 B dostupnih za program, a udio SRAM-a od ukupno 2048 B.

> **3. Pronaći na internetu biblioteku za senzor SCD30. Usporediti najmanje dvije (na primjer SparkFun, Sensirion i Adafruit) prema kriterijima iz tablice u poglavlju 7 i odabrati jednu.**

Uspoređene su biblioteke tvrtki SparkFun i Adafruit. Obje inačice programa prevedene su za istu pločicu, pa se zauzeće memorije može izravno usporediti.

Kriterij	Biblioteka 1	Biblioteka 2
Naziv i autor	SparkFun SCD30 Arduino Library, SparkFun Electronics	Adafruit SCD30, Adafruit Industries
Inačica koju ste uzeli	1.0.20	1.0.11
Datum zadnje izmjene	7. 10. 2022.	15. 11. 2023.
Broj otvorenih prijava problema	0	2
Ima li primjere i dokumentaciju	Da	Da
Licencija	MIT	BSD
Zauzeće flasha i SRAM-a	9368 B (30 %) i 456 B (22 %)	9696 B (31 %) i 515 B (25 %)
Odabrana i zašto	**Da**, jer je jednostavnija, zauzima manje memorije i treba samo ugrađenu biblioteku `Wire`	Ne, jer zauzima više memorije i uz `Wire` treba još Adafruit BusIO i Adafruit Unified Sensor

Odabrana je biblioteka SparkFun. Program s njom zauzima **328 B manje flasha** i **59 B manje SRAM-a**. Osim toga, za rad joj je dovoljna biblioteka `Wire` koja dolazi uz Arduino, dok Adafruit zahtijeva instalaciju još dviju biblioteka.

> **4. Uključiti odabranu biblioteku u projekt s točno navedenom inačicom i zabilježiti izbor i razloge u `README.md`.**

U program je uključena **SparkFun SCD30 Arduino Library**, inačica **1.0.20**. Razlozi izbora, spajanje i upute za prevođenje opisani su u datoteci `README.md`.

```
#include <Wire.h>
#include <SparkFun_SCD30_Arduino_Library.h>

SCD30 senzor;
```

Naredba `SCD30 senzor` samo stvara objekt `senzor` i pritom ne komunicira sa senzorom. Senzor se inicijalizira tek u funkciji `setup()`, pozivom `senzor.begin()`:

```
senzorSpreman = senzor.begin(Wire, false);
```

Funkcija vraća `true` ako je senzor uspješno pokrenut. Drugi argument `false` isključuje automatsku samokalibraciju senzora.

> **5. Spojiti senzor prema poglavlju 5 i napisati ispis svih adresa na I2C sabirnici. Potvrditi da se senzor javlja na 0x61.**

Senzor je spojen prema sljedećoj tablici, s napajanjem na priključku 5V i linijama SDA i SCL na pinovima A4 i A5.

Arduino Nano	SCD30	Opis
5V	VIN	Napajanje senzora s regulatora, ne s ulazno-izlaznog pina
GND	GND	Zajednička masa
A4	SDA	Podatkovna linija
A5	SCL	Taktna linija

Pri pokretanju program redom provjerava adrese od 1 do 126. Ako na nekoj adresi uređaj odgovori, funkcija `endTransmission()` vrati 0 i adresa se ispiše u heksadekadskom obliku.

```
for (uint8_t a = 1; a <= 126; a++) {
  Wire.beginTransmission(a);
  if (Wire.endTransmission() == 0) {
    Serial.print(F("Pronaden uredaj: 0x"));
    if (a < 16) Serial.print('0');
    Serial.println(a, HEX);
  }
}
```

Skeniranjem je pronađen jedan uređaj na adresi `0x61`, koja odgovara senzoru SCD30, što potvrđuje i ispis serijskog monitora pri pokretanju:

```
10:28:12.326 -> Pokretanje programa...
10:28:12.326 -> Skeniranje I2C sabirnice...
10:28:12.326 -> Pronaden uredaj: 0x61
10:28:12.326 -> SCD30 uspjesno pokrenut!
10:28:12.326 -> Slobodni SRAM pri pokretanju: 1582 B
```

> **6. Pokrenuti mjerenje i svake dvije sekunde ispisati CO_2, temperaturu i vlagu na serijski monitor.**

Program svake **2 s** provjerava ima li senzor novo mjerenje i, ako ima, očitava ga i ispisuje koncentraciju CO_2, temperaturu i relativnu vlažnost.

```
if (!senzor.dataAvailable()) {
  Serial.println(F("Podatak jos nije spreman."));
} else if (!senzor.readMeasurement()) {
  Serial.println(F("Greska pri citanju mjerenja!"));
} else {
  Serial.print((float)senzor.getCO2(), 0);
  Serial.print(senzor.getTemperature(), 2);
  Serial.print(senzor.getHumidity(), 2);
}
```

CO_2 se ispisuje kao cijeli broj, a temperatura i vlaga na dvije decimale, kao što se vidi u primjeru ispisa vrijednosti očitanih u laboratoriju T3-28:

```
10:30:00.112 -> CO2: 858 ppm | Temperatura: 22.17 C | Vlaga: 37.42 %
10:30:02.112 -> CO2: 861 ppm | Temperatura: 22.19 C | Vlaga: 37.48 %
10:30:04.113 -> CO2: 866 ppm | Temperatura: 22.22 C | Vlaga: 37.55 %
10:30:06.113 -> CO2: 864 ppm | Temperatura: 22.23 C | Vlaga: 37.61 %
10:30:08.114 -> CO2: 860 ppm | Temperatura: 22.20 C | Vlaga: 37.50 %
10:30:10.114 -> CO2: 863 ppm | Temperatura: 22.19 C | Vlaga: 37.44 %
```

Vrijednosti blago osciliraju oko prosjeka, CO_2 za nekoliko ppm, a temperatura i vlaga za nekoliko stotinki. To je očekivano jer se zrak u prostoriji stalno miješa, a promjene su manje i od točnosti senzora. Prosjek zabilježenih vrijednosti u normalnim uvjetima prikazan je u sljedećoj tablici:

Uvjeti	**CO_2 (ppm)**	**Temperatura (°C)**	**Vlaga (%)**
Prosjek u normalnim uvjetima	862	22,20	37,50

7. Ponovno očitati zauzeće memorije nakon dodavanja biblioteke i mjerenja. Izračunati koliko je postotaka SRAM-a ostalo slobodno i prokomentirati nalaz.

Zauzeće memorije očitano je iz ispisa Arduino IDE-a nakon zadatka 1 i nakon zadatka 6:

Stavka	**Vrijednost**
Zauzeće flasha nakon zadatka 1 (B i %)	1918 B (6 %)
Zauzeće SRAM-a nakon zadatka 1 (B i %)	184 B (8 %)
Zauzeće flasha nakon zadatka 6 (B i %)	9368 B (30 %)
Zauzeće SRAM-a nakon zadatka 6 (B i %)	456 B (22 %)

Dodavanjem biblioteke i mjerenja flash je porastao za 7450 B, odnosno gotovo pet puta, a SRAM za 272 B, odnosno otprilike 2,5 puta. Arduino Nano ima **2048 B SRAM-a**, a globalne varijable konačne inačice zauzimaju 456 B, pa ostaje:

$$2048 - 456 = \boxed{1592\ \text{B}}, \quad \text{odnosno} \quad \frac{1592}{2048} \cdot 100 = \boxed{77{,}73\,\%}.$$

Slobodno je dakle više od tri četvrtine SRAM-a, pa program ima dovoljno prostora za proširenje, iako dio tog prostora tijekom rada troši stog. Za usporedbu, Adafruit inačici ostaje **1533 B (74,85 %)**, što potvrđuje da SparkFun biblioteka troši manje memorije.

> **8. Preoblikovati petlju tako da ne koristi delay nego mjeri vrijeme funkcijom millis, i to tako da LED i dalje trepće dok se čeka mjerenje.**

LED-ica i mjerenje imaju svaka svoju vremensku oznaku:

```
const unsigned long sada = millis();

if (sada - tLed >= 500) {
  tLed = sada;
  ledStanje = !ledStanje;
  digitalWrite(LED_PIN, ledStanje);
}

if (sada - tMjerenje >= 2000) {
  tMjerenje = sada;
  // Provjera i citanje senzora iz zadatka 6.
}
```

Funkcija `millis()` vraća broj milisekundi od pokretanja pločice. Program u svakom prolazu petlje provjerava koliko je vremena prošlo i ništa ne čeka i LED-ica zato mijenja stanje svakih 500 ms, a senzor se provjerava svakih 2000 ms, neovisno jedno o drugome.

> **9. Dok program radi, odspojiti podatkovnu liniju. Zabilježiti ponašanje i dopuniti kod tako da program ne ostane zaglavljen nego prijavi pogrešku i nastavi.**

Za vrijeme rada programa odspojena je SDA linija, nakon čega je serijski monitor zabilježio sljedeći ispis:

```
10:44:04.952 -> CO2: 1319 ppm | Temperatura: 24.81 C | Vlaga: 49.20 %
                * odspojen SDA *
10:44:06.969 -> GRESKA: SCD30 ne odgovara!
```

Program prepoznaje prekid komunikacije, svake 2 s ispisuje grešku i nastavlja raditi. Ako senzor ne odgovori 10 s, program jednom ispisuje upozorenje da senzor ne odgovara i za to su u kod dodani I2C timeout i upozorenje:

```
// U setup(): najdulje cekanje na I2C transakciju je 25 ms.
Wire.setWireTimeout(25000, true);

// U loop(): upozorenje se ispisuje samo jednom.
if (!upozorenjeIspisano && millis() - tZadnjiOdgovor >= 10000) {
  Serial.println(F("UPOZORENJE: SCD30 ne odgovara vise od 10 s!"));
  upozorenjeIspisano = true;
}
```

Timeout ograničava čekanje na I2C sabirnici na najviše 25 ms, pa program ne ostaje zaglavljen, a LED-ica zbog zasebne vremenske oznake **nastavlja treptati i kada senzor ne odgovara**. Kada senzor ponovno odgovori, upozorenje se ponovno može ispisati.

10. Izdahnuti u blizini senzora i promatrati odziv. Zabilježiti koliko sekundi treba da se vrijednost vrati na početnu razinu.

Nakon izdaha uz senzor, koncentracija CO_2 naglo je porasla, a zatim se postupno vratila na uobičajenu razinu, kao što se vidi u ispisu serijskog monitora:

```
10:18:09.377 -> CO2: 22888 ppm | Temperatura: 22.93 C | Vlaga: 51.53 %
10:19:39.358 -> CO2: 895 ppm | Temperatura: 22.45 C | Vlaga: 36.50 %
```

Vrijeme povratka razlika je vremenskih oznaka tih dvaju ispisa:

$$10{:}19{:}39{,}358 - 10{:}18{:}09{,}377 = \boxed{89{,}981\ \text{s}}.$$

Koncentracija CO_2 porasla je na **22.888 ppm**, a na uobičajenu razinu vratila se nakon približno **90 s**. Izdah osim CO_2 povećava i vlagu, koja je porasla s oko 37 % na 51,53 %.

11. Postaviti repozitorij s datotekom `README.md` koja sadrži shemu spajanja, odabranu biblioteku s inačicom i upute za prevođenje. Predati poveznicu asistentu.

Datoteka `README.md` opisuje spajanje, odabranu biblioteku s inačicom i upute za prevođenje, a konačni program nalazi se u mapi `LV1_SCD30_SparkFun`. Poveznica na repozitorij: https://github.com/PerKan21/Primjena-mikroupravljackih-sustava.

12. Neobavezno, za one koji žele dublje: otvoriti izvorni kod odabrane biblioteke i pronaći mjesto gdje se računa provjera CRC i mjesto gdje se dvije 16-bitne riječi spajaju u broj s pomičnim zarezom.

Oba mjesta nalaze se u datoteci `SparkFun_SCD30_Arduino_Library.cpp`. Senzor svaku izmjerenu veličinu šalje kao dvije 16-bitne riječi, a iza svake riječi slijedi jedan CRC bajt.

Provjera CRC računa se u funkciji `computeCRC8()`:

```
uint8_t SCD30::computeCRC8(uint8_t data[], uint8_t len)
{
  uint8_t crc = 0xFF; // Init with 0xFF

  for (uint8_t x = 0; x < len; x++)
  {
    crc ^= data[x]; // XOR-in the next input byte

    for (uint8_t i = 0; i < 8; i++)
    {
      if ((crc & 0x80) != 0)
        crc = (uint8_t)((crc << 1) ^ 0x31);
      else
        crc <<= 1;
    }
  }

  return crc; // No output reflection
}
```

Funkcija računa CRC-8 nad dva podatkovna bajta s početnom vrijednošću `0xFF` i polinomom `0x31`. U funkciji `readMeasurement()` rezultat se uspoređuje s CRC bajtom koji je poslao senzor, a ako se razlikuju, mjerenje se odbacuje.

Spajanje dviju riječi u float odvija se u funkciji `readMeasurement()` pomoću unije `ByteToFl`, koja ista četiri bajta u memoriji može čitati kao niz bajtova ili kao broj s pomičnim zarezom:

```
typedef union
{
  uint8_t array[4];
  float value;
} ByteToFl;
```

Pristigli bajtovi obiju riječi upisuju se u niz `array` obrnutim redoslijedom, jer senzor šalje najznačajniji bajt prvi, a ATmega328P sprema najmanje značajni bajt na početak. Gotov broj zatim se čita preko člana `value`:

```
co2 = tempCO2.value;
temperature = tempTemperature.value;
humidity = tempHumidity.value;
```

Tablica za upis rezultata

Stavka	Vrijednost
Zauzeće flasha nakon zadatka 1 (B i %)	1918 B (6 %)
Zauzeće SRAM-a nakon zadatka 1 (B i %)	184 B (8 %)
Zauzeće flasha nakon zadatka 6 (B i %)	9368 B (30 %)
Zauzeće SRAM-a nakon zadatka 6 (B i %)	456 B (22 %)
Adrese pronađene na I2C sabirnici	`0x61`
Odabrana biblioteka i inačica	SparkFun SCD30 Arduino Library 1.0.20
Stvarni razmak između mjerenja (s)	2 s
Vrijednost CO_2 u mirovanju i nakon daha (ppm)	862 ppm i 22.888 ppm
Vrijeme povratka na početnu razinu (s)	89,981 s, odnosno približno 90 s
Ponašanje pri odspojenoj podatkovnoj liniji	Program svake 2 s ispisuje grešku, nakon 10 s jednom ispisuje upozorenje i nastavlja raditi, a LED-ica pritom i dalje trepće

Pitanja za provjeru znanja

> **1. Po čemu se razvojna pločica razlikuje od samog mikroupravljača i koji dijelovi pločice mogu biti uzrok kvara?**

Mikroupravljač je samo jedan čip, koji sadrži procesorsku jezgru, memoriju i periferiju. Razvojna pločica mu dodaje ono što mu treba za rad: regulator napona, izvor takta, tipkalo za reset, kondenzatore, konektore i pretvarač USB u serijsko sučelje za upis programa. Zato kvar ne mora biti u mikroupravljaču, nego može biti u bilo kojem od tih dijelova, ali i u USB kabelu ili upravljačkom programu na računalu.

2. Koje tri vrste memorije postoje na vašoj pločici i što se u koju sprema?

Memorija	Kapacitet	Što se sprema	Bez napajanja
Flash	32 KB	Program i konstante	Ostaje
SRAM	2 KB	Varijable, stog i međuspremnici	Gubi se
EEPROM	1 KB	Postavke i drugi trajni podaci	Ostaje

Od 32 KB flasha 2 KB rezervirano je za pokretački program (engl. *bootloader*), pa je za program dostupno 30.720 B.

3. Zašto se senzor SCD30 ne smije napajati s običnog ulazno-izlaznog pina?

Senzor SCD30 tijekom mjerenja troši do **75 mA**, a jedan ulazno-izlazni pin mikroupravljača smije dati najviše 40 mA, pri čemu se preporučuje ostati ispod 20 mA. Napajanje s takvog pina bilo bi nestabilno, a pin i mikroupravljač mogli bi se oštetiti. Senzor se zato napaja s priključka **5V**, koji struju dobiva s regulatora na pločici, a masa mu je zajednička s pločicom.

4. Čemu služe otpornici prema napajanju na I2C sabirnici?

Linije SDA i SCL na I2C sabirnici izvedene su tako da ih uređaji mogu samo spustiti u nisko stanje, ali ih ne mogu sami podići u visoko. Otpornici prema napajanju (engl. *pull-up*) zato vraćaju linije u visoko stanje kad ih nijedan uređaj ne drži nisko. Gotovi moduli često već imaju te otpornike, pa ih nije potrebno dodavati.

5. Zašto funkcija delay otežava kasnije dodavanje mrežne komunikacije?

Funkcija `delay()` zaustavlja program dok ne istekne zadano vrijeme, pa mikroupravljač za to vrijeme ne može raditi ništa drugo. Mrežna komunikacija zahtijeva da program stalno provjerava i obrađuje pakete koji stignu, a dok traje `delay()`, ti paketi čekaju ili se gube. Pomoću `millis()` program samo provjerava koliko je vremena prošlo i nastavlja dalje, pa između mjerenja stigne obraditi komunikaciju, LED-icu i greške.

6. Što biste morali promijeniti u današnjem sklopu i kodu da isti program radi na ESP32-S3?

Senzor treba spojiti na **3V3**, **GND**, **GPIO8 (SDA)** i **GPIO9 (SCL)**, jer ESP32-S3 radi s logičkim razinama od 3,3 V. U kodu treba promijeniti I2C i LED pinove te ukloniti ili prilagoditi procjenu slobodne memorije i I2C timeout, koji su napisani za Arduino Nano.